

Beyond CPU-Based Horizontal Pod Autoscaling: Empirical Limits of Kubernetes Autoscaling in Local and Managed Cloud Environments

Ivo Motoki Makiyama Lopes
Escola Politecnica, Universidade de Sao Paulo
PSI5120 – Topicos de Computacao em Nuvem
ivo.motoki@usp.br

Abstract—Horizontal Pod Autoscaling (HPA) is a central mechanism for elastic cloud-native applications, but increasing the desired number of replicas does not guarantee that a cluster can actually schedule and run them. This paper extends a previous Kubernetes HPA experiment by comparing the same CPU-bound web workload in local Minikube and managed Amazon EKS environments, then adding an HPA behavior-tuning experiment. The study measures reaction time, time to peak replicas, scale-down delay, effective schedulable capacity, and replica-seconds as an operational cost proxy. Preliminary results show that Minikube reaches the configured HPA maximum, while EKS can be capped below that maximum by node-level pod density and Amazon VPC CNI constraints. The paper argues that practical elasticity requires reasoning about the metrics pipeline, HPA policy, scheduler capacity, node autoscaling, and cloud cost together.

Index Terms—Kubernetes, Horizontal Pod Autoscaler, Amazon EKS, Minikube, cloud computing, autoscaling, container orchestration.

I. INTRODUCTION

Autoscaling is one of the main operational promises of cloud computing: instead of provisioning fixed capacity for peak demand, applications can adapt resource allocation to observed load. In Kubernetes, the Horizontal Pod Autoscaler (HPA) implements this mechanism by periodically changing the replica count of scalable workloads such as Deployments based on metrics such as CPU utilization [1].

In practice, however, HPA is only one control loop inside a larger system. It can request more replicas, but it cannot by itself guarantee that the cluster has enough schedulable capacity, IP addresses, node resources, or cloud budget to run them. This distinction is especially important in managed Kubernetes environments such as Amazon EKS, where pod density depends not only on CPU and memory but also on instance type and Amazon VPC CNI networking limits [2].

This work extends an intermediate PSI5120 project that deployed the official Kubernetes CPU-bound HPA example workload in both Minikube and Amazon EKS. The final project has three goals: first, to quantify the baseline behavior already observed in both environments; second, to add a controlled experiment using explicit `autoscaling/v2` behavior policies; and third, to analyze why effective elasticity in EKS can be bounded by infrastructure constraints outside the HPA object itself.

The guiding research questions are:

- RQ1: With the same workload and HPA target, how do Minikube and EKS differ in reaction time, peak replicas, and scale-down behavior?
- RQ2: When HPA recommends additional replicas in EKS, which infrastructure limits can prevent those replicas from becoming ready?
- RQ3: How do explicit HPA behavior policies change underprovisioning and overprovisioning windows compared with Kubernetes defaults?

II. BACKGROUND

A. Horizontal Pod Autoscaler

The Kubernetes HPA controller periodically computes a desired replica count from observed metrics. For resource metrics such as CPU, the controller compares current utilization with the configured target and conceptually applies the ratio between the current and desired values [1]. CPU utilization is expressed relative to each container's `resources.requests.cpu`, so missing or poorly chosen requests can make HPA behavior undefined or misleading.

The default HPA control loop favors fast scale-up and conservative scale-down. Kubernetes documents a default 15 s synchronization period for the controller and a default 300 s stabilization window for scale-down recommendations [1]. The `autoscaling/v2` API exposes a `behavior` field that allows experiments with explicit scale-up and scale-down policies, including stabilization windows, pod-based policies, percent-based policies, and policy selection rules [3].

B. Metrics Pipeline

The HPA does not collect resource metrics directly. It relies on the Kubernetes metrics APIs, commonly backed by Metrics Server. Metrics Server is intended as an efficient source for autoscaling resource metrics, not as a general-purpose monitoring or historical observability system [4]. This distinction motivates the use of experiment logs and derived CSV data in this project.

C. Amazon EKS Capacity Constraints

Amazon EKS runs standard Kubernetes HPA behavior, provided that a metrics source such as Metrics Server is

available [5]. The managed cloud environment adds other constraints. With the Amazon VPC CNI plugin in the default mode, pods receive VPC IP addresses, and pod density is bounded by the number of elastic network interfaces and IP addresses supported by each EC2 instance type [2]. Therefore, a workload may hit scheduling limits even when the HPA `maxReplicas` value is higher.

III. RELATED WORK

Recent work has evaluated HPA in microservice environments, including throughput, latency, resource utilization, and operational cost trade-offs [6]. Other studies have proposed finer-grained horizontal scaling policies to address delayed response and conservative anti-oscillation behavior in container clouds [7]. This paper is narrower in workload complexity, but contributes a reproducible educational comparison between a local Kubernetes cluster and a managed cloud cluster under realistic infrastructure limits.

IV. EXPERIMENTAL SETUP

A. Workload

Both environments run the official Kubernetes HPA walk-through workload, `registry.k8s.io/hpa-example`, exposed through a `php-apache` Deployment and Service [8]. The container requests 200 mCPU and 64 MiB of memory, with limits of 500 mCPU and 128 MiB. The baseline HPA targets 50% average CPU utilization with `minReplicas: 1` and `maxReplicas: 10`.

B. Load Generation

Load is generated from inside the cluster using BusyBox pods repeatedly issuing HTTP requests to the internal Kubernetes Service DNS name. This avoids differences caused by external network exposure and keeps the stress method comparable between Minikube and EKS.

C. Measurements

The experiment logs `kubectl get hpa`, `kubectl get pods -o wide`, and `kubectl top pods` every few seconds during active load and after load removal. The final project derives the following metrics from raw logs: time to first scale-up, time to peak replicas, time to scale-down, peak desired replicas, running workload pods, pending workload pods, replica-seconds, and time spent above the CPU target.

V. BASELINE RESULTS

The baseline analysis uses the raw timeline logs produced in the intermediate project. Each run records HPA state, pod state, and resource metrics during the active-load window and after load removal. Table I summarizes the metrics extracted automatically from those logs.

The parsed data confirms the qualitative findings from the intermediate report. In the Minikube baseline, HPA reached the configured maximum of 10 replicas without pending workload pods. In the EKS runs, the HPA desired replica count reached

TABLE I
BASELINE HPA METRICS EXTRACTED FROM RAW EXPERIMENT LOGS

Environment	First scale-up (s)	Peak replicas	Pending pods	Scale-down after load (s)	Replica-s
Minikube	82	10	0	144	2450
EKS (005559)	230	5	4	—	1983
EKS (010657)	50	5	4	351	2773

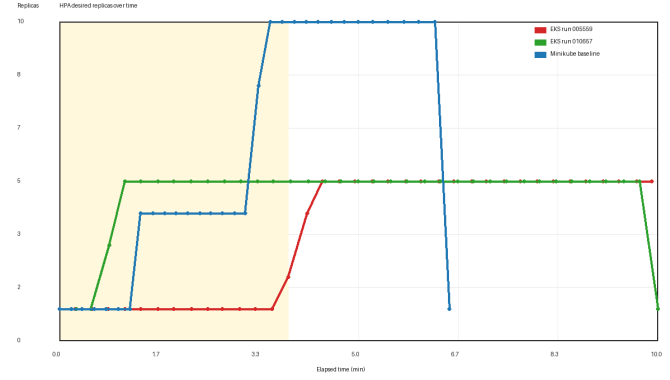


Fig. 1. HPA desired replicas over time for the existing baseline runs. The shaded area indicates the active-load phase of the first parsed run.

only 5, while up to 4 `php-apache` pods remained pending. This separates the HPA decision from effective workload capacity: the controller can request replicas, but the scheduler can only place them if the cluster has enough node and networking capacity.

VI. HPA BEHAVIOR-TUNING EXTENSION

The extension compares the default HPA behavior against two explicit `autoscaling/v2` policies: an aggressive scale-up policy and a faster scale-down policy. The expected trade-off is that faster scale-up may reduce the underprovisioning window, while faster scale-down may reduce replica-seconds after a burst but increase the risk of oscillation for intermittent workloads.

VII. CLOUD INFRASTRUCTURE CONSTRAINTS

The EKS baseline is used to show the difference between configured elasticity and effective elasticity. HPA may set a higher desired replica count, but the scheduler can leave pods pending if nodes lack capacity. In the tested EKS configuration, `t3.micro` nodes are particularly constrained: the instance family is burstable, has limited memory, and, more importantly for this experiment, pod density is bounded by VPC CNI IP address capacity [2], [9].

VIII. COST AND OPERATIONAL DISCUSSION

Minikube has no direct cloud cost, but it depends on local host capacity and is not a production-grade managed environment. EKS introduces a managed control-plane charge and separate costs for worker nodes and related AWS resources [10]. This section will estimate the cost of reproducing the experiment and discuss when HPA must be combined with node autoscaling mechanisms such as Karpenter or Cluster Autoscaler [11].

IX. THREATS TO VALIDITY

The workload is intentionally simple and CPU-bound, so results should not be generalized to all microservice or I/O-bound systems. The EKS experiment used small Free-Tier-constrained instances, which makes infrastructure constraints visible but does not represent all production EKS clusters. Finally, Metrics Server values are sufficient for HPA control but are not a high-resolution observability dataset.

X. CONCLUSION

The final project extends a working HPA comparison into a broader empirical study of Kubernetes elasticity. The expected conclusion is that HPA is necessary but insufficient: practical autoscaling requires aligning application metrics, controller behavior, schedulable capacity, node autoscaling, and cloud cost.

REFERENCES

- [1] Kubernetes Authors, “Horizontal pod autoscaling,” <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>, 2026, accessed: 2026-09-06.
- [2] Amazon Web Services, “Amazon vpc cni,” <https://docs.aws.amazon.com/eks/latest/best-practices/vpc-cni.html>, 2026, accessed: 2026-09-06.
- [3] Kubernetes Authors, “Horizontalpodautoscaler v2 api reference,” <https://kubernetes.io/docs/reference/kubernetes-api/autoscaling/horizontal-pod-autoscaler-v2/>, 2026, accessed: 2026-09-06.
- [4] Kubernetes SIGs, “Metrics server,” <https://kubernetes-sigs.github.io/metrics-server/>, 2026, accessed: 2026-09-06.
- [5] Amazon Web Services, “Scale pod deployments with horizontal pod autoscaler,” <https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>, 2026, accessed: 2026-09-06.
- [6] H. Buzato and A. Goldman, “Extended version of microservices performance optimization through horizontal pod autoscaling: A comprehensive study,” *International Journal of Networking and Computing*, vol. 16, no. 1, pp. 52–74, 2026. [Online]. Available: https://www.jstage.jst.go.jp/article/ijnc/16/1/16_52/_article/-char/en
- [7] Q. Jiang and Y. Wu, “A fine-grained horizontal scaling method for container-based cloud,” *Scientific Programming*, 2021. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1155/2021/6397786>
- [8] Kubernetes Authors, “Horizontal pod autoscaling walkthrough,” <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>, 2026, accessed: 2026-09-06.
- [9] Amazon Web Services, “Amazon ec2 t3 instances,” <https://aws.amazon.com/ec2/instance-types/t3/>, 2026, accessed: 2026-09-06.
- [10] —, “Amazon eks pricing,” <https://aws.amazon.com/eks/pricing/>, 2026, accessed: 2026-09-06.
- [11] —, “Scale cluster compute with karpenter and cluster autoscaler,” <https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html>, 2026, accessed: 2026-09-06.